

Student Report

Achilleas Villiotis - 1084567

Implementation Challenges

The primary challenge encountered during the implementation was designing an architecture that balances scalability, fault tolerance, and performance. The choice between using a distributed framework like Apache Kafka and Spark versus more centralized solutions required careful consideration. Apache Kafka and Spark were chosen for their ability to handle high-throughput, real-time data processing and their robust ecosystem for integration with other tools.

Another significant challenge was resolving the `java.lang.NoClassDefFoundError: scala/$less$colon$less` error. This issue arose due to missing or incompatible Kafka-Spark connector dependencies. Identifying the correct versions of the required JAR files and ensuring compatibility with Spark 3.5.0 required extensive research and testing. Additionally, configuring the Docker environment to include these dependencies posed its own set of challenges, particularly in ensuring that the updated image was correctly built and deployed.

System Performance Analysis

The system demonstrated robust performance after resolving the dependency issues. The Kafka-Spark streaming application was able to process a high volume of messages with low latency, indicating efficient data handling and processing capabilities. Resource utilization metrics were within acceptable limits, suggesting that the system is well-optimized for the current workload. However, it is worth noting that performance may vary under different operational conditions or with increased data loads.

ML Model

Because of time constraints, the model was trained with the current weather instead of a 1-hour forecast. This obviously degrades the model performance, but it was a necessary compromise to ensure the project could be completed on time.

Potential Improvements

1. **Scalability:** While the system performed well under the current workload, future improvements could focus on enhancing scalability to handle larger data volumes and higher message throughput. The design followed already allos for most of the scalability needs, but further testing and optimization could be beneficial.
2. **Monitoring and Alerting:** Implementing a comprehensive monitoring and alerting system would help in proactively identifying and addressing potential issues.
3. **Dependency Management:** Automating the process of dependency management and version compatibility checks could prevent similar issues in the future. Especially not using the uv package manager for creating the requirements.txt file, but instead creating it via a more standard approach.
4. **Resource Optimization:** Further optimization of resource usage, particularly memory and CPU, could improve overall system efficiency.